

Cas Cremers

Scyther

User Manual

Version 1.3.0

February 18, 2026

Contents

1	Introduction	7
2	Background	9
3	Download and Installation	11
4	Quick start tutorial	13
5	Input Language	17
5.1	A minimal input file	17
5.2	Terms	17
5.2.1	Atomic terms	17
5.2.2	Pairing	18
5.2.3	Symmetric keys	18
5.2.4	Asymmetric keys	18
5.2.5	Hash functions	19
5.2.6	Predefined types	19
5.2.7	Usertypes	19
5.3	Events	19
5.3.1	Receive and send events	19
5.3.2	Claim events and Security properties	20
5.3.3	Internal computation/pattern match events	20
5.4	Role definitions	21
5.5	Protocol definitions	21
5.6	Global declarations	22
5.7	Miscellaneous	23
5.7.1	Macro	23
5.7.2	Include	23
5.7.3	one-role-per-agent	24
5.8	Language BNF	24
5.8.1	Input file	24
5.8.2	Protocols	24
5.8.3	Roles	24
5.8.4	Events	24
5.8.5	Declarations	25
5.8.6	Terms	25
6	Modeling security protocols	27
6.1	Introduction	27
6.2	Example: Needham-Schroeder Public Key	28

7	Specifying security properties	31
7.1	Specifying secrecy	31
7.2	Specifying authentication properties	31
7.2.1	Aliveness	31
7.2.2	Non-injective synchronisation	32
7.2.3	Non-injective agreement	32
7.2.4	Agreement over data	32
8	Using the Scyther tool GUI	33
8.1	Results	33
8.2	Bounding the statespace	34
8.3	Attack graphs	35
8.3.1	Visual Notation Reference	35
8.3.2	Runs	38
8.3.3	Communication events	39
8.3.4	Claims	39
8.4	GUI Settings and Configuration	39
8.4.1	Verification Parameters	40
8.4.2	Verification Modes	40
8.4.3	Performance Tuning	40
8.5	GUI Menu Reference	41
8.5.1	File Menu	41
8.5.2	Verify Menu	41
8.5.3	Working with Multiple Protocols	41
8.5.4	Exporting Results	41
8.5.5	Tips for Effective Use	42
9	Using the Scyther command-line tools	43
9.1	Python wrapper scripts	43
9.1.1	scyther.py: Simple command-line wrapper	44
9.1.2	scyther-gui.py: GUI with command-line options	44
9.1.3	json-scyther.py: Batch processing interface	45
10	Advanced topics	47
10.1	Modeling more than one asymmetric key pair	47
10.2	Approximating equational theories	47
10.3	Modeling time-stamps and global counters	49
10.3.1	Modeling global counters	49
10.3.2	Modeling time-stamps using nonces	49
10.3.3	Modeling time-stamps using variables	49
10.4	Multi-protocol attacks	50
11	Troubleshooting and FAQ	51
11.1	Installation and Setup Issues	51
11.1.1	macOS: “Binary cannot be opened because the developer cannot be verified”	51
11.1.2	GUI fails to start or shows blank windows	51
11.1.3	Command not found or Python errors	51
11.2	Understanding Verification Results	52
11.2.1	What’s the difference between “No attacks within bounds” and “No attacks”?	52
11.2.2	Verification is taking too long	52
11.2.3	Why did Scyther find an attack I don’t think is realistic?	52
11.3	Protocol Modeling Issues	52
11.3.1	No attacks found but I expected one	52
11.3.2	Parse errors and syntax issues	53

11.3.3	Unexpected variable bindings	53
11.4	Common Questions	53
11.4.1	Should I use automatic claims or write my own?	53
11.4.2	Can Scyther verify protocols with Diffie-Hellman or other cryptographic operations?	54
11.4.3	How do I verify a protocol with more than two roles?	54
11.4.4	Where can I find more example protocols?	54
A	Full example specification for Needham-Schroeder public key	57

Chapter 1

Introduction

This is the user manual for the Scyther security protocol verification tool, and complements the website [5], github repository [4], and the accompanying book [2].

The target audience of Scyther is anyone interested in learning about the formal analysis of security protocols, including researchers, students, and practitioners. The tool is designed to be easy to use for beginners. However, it does not offer state-of-the-art features, and its analysis is relative coarse for modern standards. Thus, while it can be extremely useful for teaching purposes or finding basic attacks, it is not recommended to use Scyther for the analysis of complex protocols or for showing the absence of subtle attacks. For such more advanced analysis and to obtain substantially higher assurance guarantees, we suggest researchers and advanced students to use the Tamarin Prover [1,9] instead, which is a far more powerful tool.

The purpose of this manual is to explain the details of the Scyther input language, explain how to model basic protocols, and how to effectively use the Scyther tool. This manual does not detail the protocol execution model nor the adversary model used by the tool. It is therefore highly recommended to read the book [2]. The book includes a detailed description of Scyther’s underlying protocol model, security property specifications, and the algorithm.

A short historical note on Scyther’s origins and current status: Scyther’s development started in 2002, and the tool has been used in many research papers since then. Since about 2012, the tool is still being maintained, but no major new features are planned. The goal is to maintain the tool to support educators and existing users, but for new research projects we strongly recommend to use the Tamarin Prover instead.

Outline

We proceed in the following way. We give some background in Chapter 2. Chapter 3 explains how to install the Scyther tool on various platforms. In Chapter 4 we give a brief tutorial using simple examples to show the basics of the tool. Then we discuss things in more detail as we introduce the input language of the tool in Chapter 5. Modeling of basic protocols is described in Chapter 6, and Chapter 7 describes how to specify security properties. The usage of the GUI version of tool is then explained in more detail in Section 8. The underlying command-line tool is described in Section 9. Advanced topics are discussed in Section 10.

Chapter 2

Background

Scyther is a tool for the formal analysis of security protocols under the *perfect cryptography assumption*, in which it is assumed that all cryptographic functions are perfect: the adversary learns nothing from an encrypted message unless he knows the decryption key. The tool can be used to find problems that arise from the way the protocol is constructed. This problem is undecidable in general, but in practice many protocols can be either proven correct or attacks can be found.

The full protocol model, its assumptions, the basic security properties, and the algorithm are described in [2]. This manual serves as a companion to the book. Thus, in this manual we assume the reader is familiar with the formal modeling of security protocols and their properties.

Chapter 3

Download and Installation

Scyther can be downloaded from the Scyther website [5] or github [4]. Installation instructions are included in the downloadable Scyther archives. Scyther is available for Windows, Linux, and Mac OS platforms. For building from source, including prerequisites and build options, see the `README.md` file in the project repository.

Chapter 4

Quick start tutorial

Scyther takes as input a security protocol description that includes a specification of intended security properties, referred to as security claims, and evaluates these.

Start Scyther by executing the `scyther-gui.py` program in the Scyther directory. The program will launch two windows: the main window, in which files are edited, and the **about** window, which shows some information about the tool.

Alternatively, you can load a protocol file directly when launching the GUI:

```
python3 scyther-gui.py protocol.spdl
```

This will open the specified file immediately in the main window. You can also suppress the splash screen for faster startup:

```
python3 scyther-gui.py --no-splash protocol.spdl
```

As an introductory example, we will verify the Needham-Schroeder protocol, and investigate an attack on it.

Go to the file→open dialog, and open the file `ns3.spdl` in the Scyther directory (or launch the GUI with `python3 scyther-gui.py ns3.spdl` to load it directly). Your main window should look like the one in Figure 4.

By convention, protocol description files have the extension `.spdl` (Security Protocol Description Language), but it can have any name. The file used in this example is shown in Appendix A.

Run the verification tool by selecting `verify→verify_claims` in the menu. A new window will appear during the verification process. Once verification is completed, the window will be replaced by the result window, as shown in Figure 4.

The result window shows a summary of the claims in the protocol, and the verification results. Here one can find whether the protocol is correct, or false. In the next section there will be a full explanation of the possible outcomes of the verification process. Most importantly, if a protocol claim is incorrect, then there exists at least one attack on the protocol. A button is shown next to the claim: press this button to view the attacks on the claim, as in Figure 4.

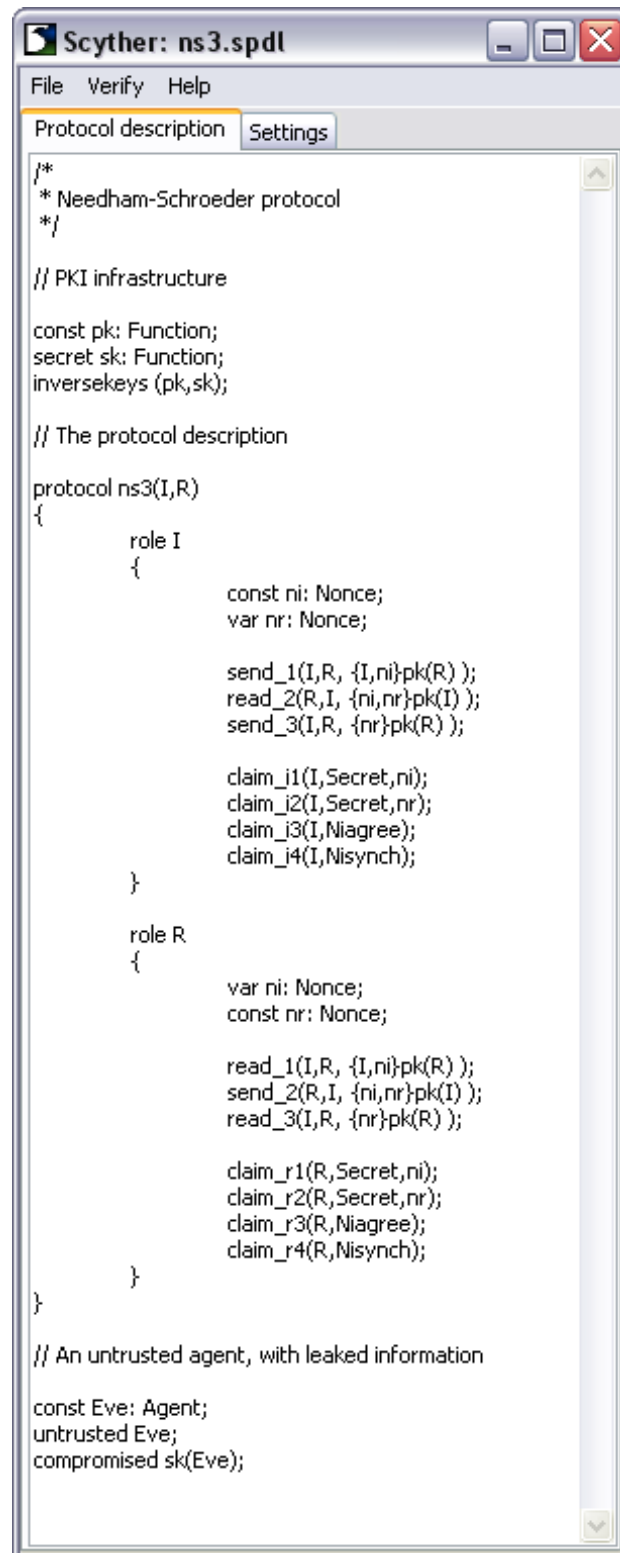


Figure 4.1: Scyther main window with the file ns3.spdl opened



The image shows a window titled "Scyther results : verify". It contains a table with four columns: Claim, Status, Comments, and Classes. The table lists results for two claims, 'ns3' and 'R'. Claim 'ns3' has four sub-claims (ns3,i1 to ns3,i4) all with 'Ok' status and 'Verified' comments. Claim 'R' has four sub-claims (ns3,r1 to ns3,r4) all with 'Fail' status and 'Falsified' comments, each with a button labeled '1 attack'. The bottom of the window shows 'Done.'.

Claim				Status	Comments	Classes
ns3	I	ns3,i1	Secret ni	Ok	Verified	No attacks.
		ns3,i2	Secret nr	Ok	Verified	No attacks.
		ns3,i3	Niagree	Ok	Verified	No attacks.
		ns3,i4	Nisynch	Ok	Verified	No attacks.
R		ns3,r1	Secret ni	Fail	Falsified	At least 1 attack. 1 attack
		ns3,r2	Secret nr	Fail	Falsified	At least 1 attack. 1 attack
		ns3,r3	Niagree	Fail	Falsified	At least 1 attack. 1 attack
		ns3,r4	Nisynch	Fail	Falsified	At least 1 attack. 1 attack

Done.

Figure 4.2: Scyther result window

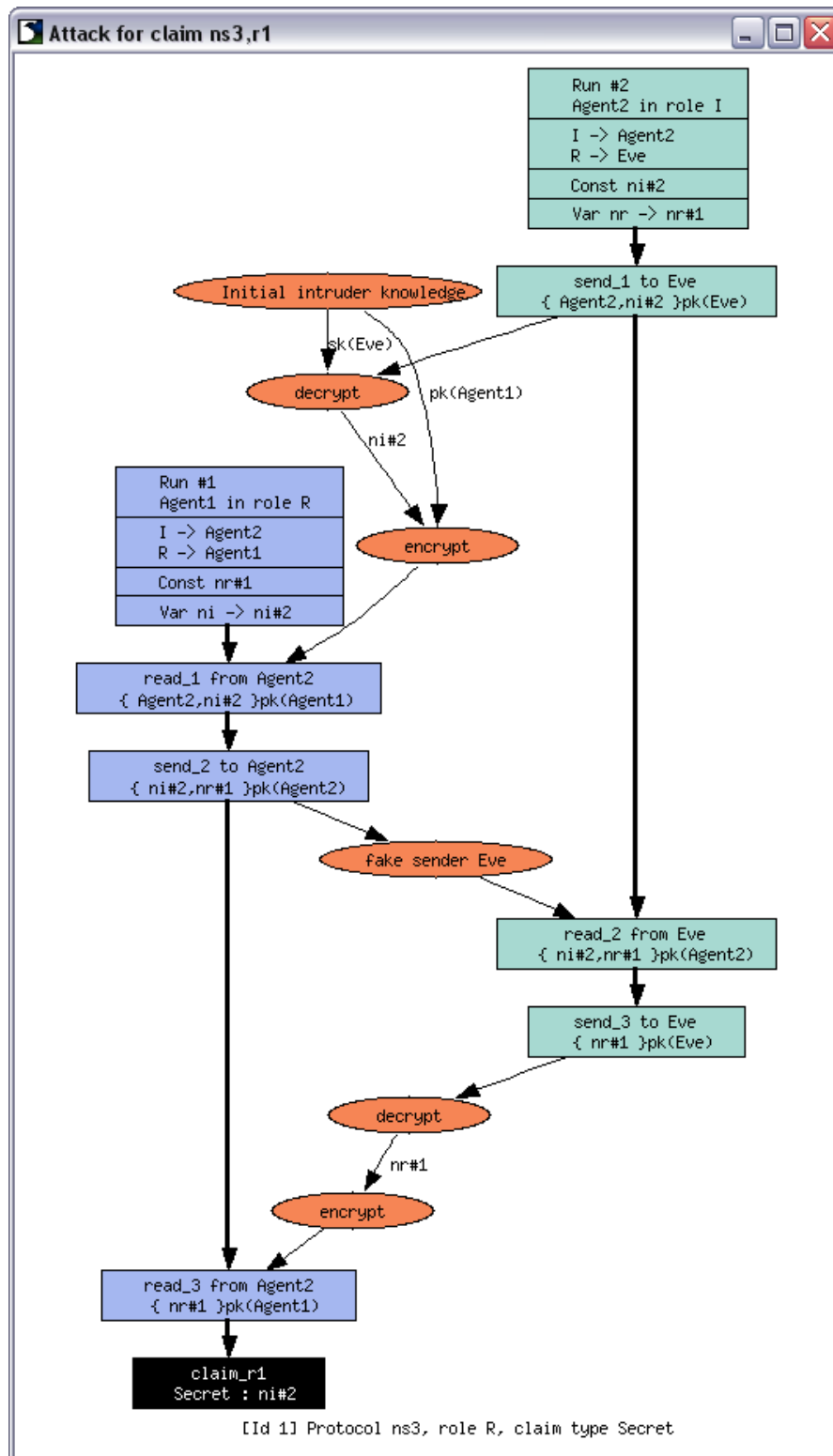


Figure 4.3: Scyther attack window

Chapter 5

Input Language

Scyther’s input language is loosely based on a C/Java-like syntax. The main purpose of the language is to describe protocols, which are defined by a set of roles. Roles, in turn, are defined by a sequence of events, most of which are events that denote the sending or receiving of terms. We describe these elements in the following sections.

Comments can start with `//` or `#` (for single-line comments) or be enclosed by `/*` and `*/` (for multi-line comments). Note that multi-line comments cannot be nested.

Any whitespace between elements is ignored. It is therefore possible to use whitespace (spaces, tabs, newlines) to improve readability.

A basic identifier consists of a string of characters from the set of alphanumeric characters as well as the symbols `^` and `-`.

The language is case-sensitive, thus `NS3` is not the same identifier as `ns3`.

5.1 A minimal input file

The core elements in a Scyther input file are protocol definitions. A minimal example is the following:

```
protocol ExampleProtocol(I,R) {  
  role I { };  
  role R { };  
};
```

In the above, we have defined a protocol called “ExampleProtocol” that has two roles, “I” and “R” by listing them between brackets after the protocol name. Note that we haven’t defined the behaviour of these roles yet: such behaviours are defined within the curly brackets after the corresponding `role I` and `role R` commands.

5.2 Terms

At the most basic level, Scyther manipulates terms.

5.2.1 Atomic terms

An atomic term can be any identifier, which is usually a string of alphanumeric characters.

Atomic terms can be combined into more complex terms by operators such as pairing and encryption.

Constants

Freshly generated values

Many security protocols rely on generating random values. They can be specified by declaring them inside a role definition using the `fresh` declaration. For example, to generate a random value `Na` of type `Nonce`, we specify:

```

role X(...) {
  fresh Na: Nonce;

  send_1(X,Y,Na);
}

```

Variables

Agents can use variables to store received terms. For example, to receive a nonce into a variable with name `Na`, we specify:

```

role Y(...) {
  var Na: Nonce;

  recv_1(X,Y,Na);
}

```

Local declarations, for both freshly generated values as well as variables such as `Na`, are local to the role. Thus, one can specify a *freshly generated nonce* `Na` in one role and a variable `Na` in another role without any conflicts. Variables are rigid: after the first receive event in which they occur has been executed, they are assigned a value. This value cannot be changed afterwards.

Variables must occur first in receive events: it is not allowed to use uninitialized variables in send events.

5.2.2 Pairing

Any two terms can be combined into a term pair: we write (x,y) for the pair of terms x and y . It is also allowed to write n -tuples as (x,y,z) , which is interpreted by Scyther as $((x,y),z)$.

5.2.3 Symmetric keys

Any term can act as a key for symmetrical encryption.

The encryption of `ni` with a term `kir` is written as:

```
{ ni }kir
```

Unless `kir` is explicitly defined as being part of an asymmetric key pair (explained below), this is interpreted as symmetric encryption.

A symmetric-key infrastructure is predefined: $k(X,Y)$ denotes the long-term symmetric key shared between X and Y .

5.2.4 Asymmetric keys

A public-key infrastructure (PKI) is predefined: $sk(X)$ denotes the long-term private key of X , and $pk(X)$ denotes the corresponding public key.

As an example, consider the following term. It represents the encryption of some term `ni` by the term `pk(I)`. Under normal conventions, this means that the nonce of the initiator (`ni`) is encrypted with the public key of the initiator.

```
{ ni }pk(I)
```

This term can only be decrypted by an agent who knows the secret key $sk(I)$.

Section 10.1 describes how to model more than one key pair per agent.

5.2.5 Hash functions

Hash functions are essentially encryptions with a function, of which the inverse is not known by anybody.

They can be used by a global declaration of an identifier to be a hashfunction, e. g.:

```
hashfunction H1;
```

As all agents and protocols should have access to such a function, the declaration of hashfunction is usually global, i. e., defined outside of any protocol definition.

Once declared, they can be used in protocol messages, e. g.:

```
H1(ni)
```

5.2.6 Predefined types

Agent Type used for agents.

Function A special type that defines a function term that can take a list of terms as parameter. By default, it behaves like a hash function: given the term $h(x)$ where h is of type **Function**, it is impossible to derive x .

Nonce A standard type that is often used and therefore defined inside the tool.

Ticket A variable of type **Ticket** can be substituted by any term.

5.2.7 Usertypes

It is possible to define a new type. This can be done using the **usertype** command:

```
usertype MyAtomicMessage;

protocol X(I,R) {
  role I {
    var y: MyAtomicMessage;

    recv_1(I,R, y );
```

The effect of such a declaration is that variables of the new type can only be instantiated with messages m of that type, i. e., that have been declared by the global declaration `const m: MyAtomicMessage` or the freshly generated `fresh m: MyAtomicMessage` within a role.

In general, the tool can perform better if more is known about which messages might unify or not. By defining a usertype, the modeler can inform the tool that a variable can only be instantiated with terms of that type, and not with, e. g., terms of type **Nonce**. Conceptually, one can always write **Ticket** (which corresponds to all possible messages) for each variable type, but then one may find false attacks (in case the implementation in fact does check the type of a message) and the tool will be less likely to verify the property (for an unbounded number of runs).

5.3 Events

5.3.1 Receive and send events

The **recv** and **send** events mark receiving and sending a message, respectively. For example, we write:

```
role MyRole(...) {
  recv_Label1(OtherRole, MyRole, m1);
  send_Label2(MyRole, OtherRole, m2);
}
```

to specify that role `MyRole` first receives message `m1` from `OtherRole` and then sends message `m2` to `OtherRole`. The receive event is labeled with label `Label1` and the send event is labeled with `Label2`.

Usually each `send` event will have a corresponding `recv` event. We specify this correspondence by giving corresponding events the same label.

```
role MyRole(...) {
  send_Label3(MyRole, OtherRole, m2);
}
role OtherRole(...) {
  recv_Label3(MyRole, OtherRole, m2);
}
```

Bang prefix for labels

For some protocols we may want to model sending or receiving to the adversary directly, in which case we have no corresponding event. If a `send` or `recv` event has no corresponding event, Scyther will output a warning. To suppress this warning, the label can be prefixed by a bang `!`, e.g.:

```
send_!1(I,I, LeakToAdversary );
```

5.3.2 Claim events and Security properties

Claim events are used in role specifications to model intended security properties. For example, the following claim event models that `Ni` is meant to be secret.

```
claim(I, Secret, Ni);
```

There are several predefined claim types.

Secret This claim requires a parameter term. Secrecy of this term is claimed as defined in [2].

Alive Aliveness (of all roles) as defined in [8].

Weakagree Weak agreement (of all roles) as defined in [8].

Commit, Running Non-injective agreement with a role on a set of data items [8] can be defined by inserting the appropriate signal claims. In this context, **Commit** marks the effective claim, whose correctness requires the existence of a corresponding **Running** signal in the trace.

These claims are used to model agreement over data, which is explained in Section 7.2.4.

Nisynch Non-injective synchronisation as defined in [2,3].

Niagree Non-injective agreement on messages as defined in [2].

Reachable When this claim is verified, Scyther will check whether this claim can be reached at all. It is true iff there exists a trace in which this claim occurs. This can be useful to check if there is no obvious error in the protocol specification, and is in fact inserted when the `--check` mode of Scyther is used.

Empty This claim will not be verified, but simply ignored. It is only useful when Scyther is used as a back-end for other verification means. For more on this, see Section 10.

5.3.3 Internal computation/pattern match events

We extend the basic set of events from [2] with two events that can be used to model internal computations.

Match event

The first new event is the `match` event, that is used to specify pattern matching, i. e.,

```
match(pt,m)
```

In operational terms, if there exists a well-typed substitution σ such that $\sigma pt = m$, then this event can be executed. Upon execution, the substitution is applied to the remaining events of the role.

This event can be used to model various constructions, such as equality tests, delayed decryption, checking commitments. They can also be used to model internal computations to simplify specifications, e. g.:

```
var X: Nonce;
var Y;

recv(R,I, X);
match(Y, hash(X,I,R) );
send(I,R, Y,{ Y }sk(I) );
```

In the above example, we could have replaced `Y` by `hash(X,I,R)` throughout the specification, but this version avoid replication.

Not match event

The second new event is the `not match` event, that is used to specify pattern matching, i. e.,

```
not match(pt,m)
```

The operational interpretation is the opposite of the previous event. If there is no substitution σ such that $\sigma pt = m$, then the event can be executed.

This event can be used to model, e. g., inequality constraints. For example, the execution model allows by default agents executing sessions with themselves. In some cases, we want to exclude such behaviour, because the protocol disallows it. For example,

```
role A {
  not match(A,B);
  send (A,B, m1);
}
```

models a role whose instances only send messages to other agents.

As a more advanced usage, `match` and `not match` can be used together in two roles with a common starting sequence of events to model *if ... then ... else* constructions.

5.4 Role definitions

Role definitions are sequences of events, i. e., declarations, send, receive, or claim events.

```
role Server {
  var x,y,z: Nonce;
  fresh n,m: Nonce;

  send_1(Server,Init, m,n );
  recv_2(Init,Server, x,y, { z }pk(Server) );
}
```

5.5 Protocol definitions

A protocol definition takes as a parameter a sequence of roles, which are then defined within its body.

```

protocol MyProt(Init,Resp,Server)
{
  role Init {
    ...
  }
  role Resp {
    ...
  }
  role Server {
    ...
  }
}

```

Helper protocols

It is possible to prepend an “@” symbol before a protocol name. This has no effect on the protocol model, nor on the outcome of the analysis. The “@” is only used when rendering output graphs: the intent is to mark the protocol as a “helper protocol”. Such protocols are often used to model additional adversary capabilities, see Section 10 for examples. When rendering output graphs, Scyther collapses role instances of helper protocols into single nodes. This can make the graphs more readable.

Symmetric-role protocols

Some adversary-compromise rules, such as *SKR* and *LKR_{aftercorrect}* depend on a partnering function. For protocols that are entirely symmetric in their roles and key computations (such as *HMQR*), this is not the appropriate partnering function. To use the correct partnering function, the protocol needs to be annotated as a *symmetric-role* protocol. This instructs Scyther to use the appropriate partnering function.

```

symmetric-role protocol MyProt(Init,Resp)
{
  role Init {
    ...
  }
  role Resp {
    ...
  }
}

```

5.6 Global declarations

In many applications global constants are used. These include, for example, string constants, labels, or protocol identifiers.

They are modeled and used in the following way:

```

usertype String;
const HelloWorld: String;

protocol hello(I,R)
{
  role I {
    send_1(I,R, HelloWorld);
  }
  role R {
    recv_1(I,R, HelloWorld);
  }
}

```

5.7 Miscellaneous

5.7.1 Macro

It is possible to define *macros*, i.e., abbreviations for particular term. The syntax used to define these abbreviations is the following:

```
macro MyShortCut = LargeTerm;
```

For example, for a protocol that contains complex messages or repeating elements, macros can be used to simplify the protocol specification:

```
hashfunction h;

protocol macro-example-one(I,R) {
  role I {
    fresh nI: Nonce;
    macro m1 = h(I,ni);

    send_1(I,R, { m1 }pk(R) );
    claim(I, Secret, m1);
  }
  role R {
    var X: Ticket;

    recv_1(I,R, { X }pk(R) );
  }
}
```

Note that macros have *global scope*, and are handled at the *syntactical* level. This also allows for global abbreviations of protocol messages, e.g.:

```
hashfunction h;
macro m1 = { I,R, nI, h(nI,R) }pk(R);

protocol macro-example-two(I,R) {
  role I {
    fresh nI: Nonce;

    send_1(I,R, m1 );
  }
  role R {
    var nI: Nonce;

    recv_1(I,R, m1 );
  }
}
```

Note that in the above example, *nI* is a freshly generated nonce in the I role, and a variable in the R role. Because the macro definitions are unfolded syntactically, the same macro can be used to refer to both terms.

5.7.2 Include

It is possible to import other files in a protocol specification:

```
include "filename";
```

where *filename* denotes the name of the file that will be included at this point. Using this command, it is possible to share, e.g., a set of common definitions between files. Typically this will include definitions for the key structures, and (untrusted) agent names. Nested use of this command is possible.

5.7.3 one-role-per-agent

The operational semantics allow agents to perform any roles, and even multiple different roles in parallel. This modeling choice corresponds to the worst possible scenario, in which the adversary has the most options to exploit. However, in many concrete settings, agents perform only one role. For example, the set of servers may be disjoint from the set of clients, or the set of RFID tags may be disjoint from the set of readers. In such cases, we do not need to consider attacks that exploit that an agent can perform multiple roles. This can be modeled by the following statement:

```
option "--one-role-per-agent"; // disallow agents in multiple roles
```

This causes Scyther to ignore attacks in which agents perform multiple roles. Phrased differently, this corresponds to the situation in which each role is performed by a dedicated set of agents.

5.8 Language BNF

The full BNF grammar for the input language is given below. In the strict language definition, there are no claim terms such as **Niagree** and **Nisynch**, and neither are there any predefined type classes such as **Agent**. Instead, they are predefined constant terms in the Scyther tool itself.

5.8.1 Input file

An input file is simply a list of spdl constructions, which are global declarations or protocol descriptions.

$$\langle \text{spdlcomplete} \rangle ::= \langle \text{spdl} \rangle \{ \text{' ;' } \langle \text{spdl} \rangle \}$$

$$\begin{aligned} \langle \text{spdl} \rangle &::= \langle \text{globaldeclaration} \rangle \\ &| \langle \text{protocol} \rangle \end{aligned}$$

5.8.2 Protocols

A protocol is simply a container for a set of roles. Because we use a role-based approach to describing roles, the protocol container in fact only affects the naming of the roles: a role “I” in a protocol “ns3” will internally be assigned the name “ns3.I”. This is used to make role names globally unique.

$$\langle \text{protocol} \rangle ::= \text{'protocol'} \langle \text{id} \rangle \text{'('} \langle \text{termlist} \rangle \text{')' '{' } \langle \text{roles} \rangle \text{'}' [\text{' ;' }]$$

5.8.3 Roles

$$\begin{aligned} \langle \text{roles} \rangle &::= \langle \text{role} \rangle [\langle \text{roles} \rangle] \\ &| \langle \text{declaration} \rangle [\langle \text{roles} \rangle] \end{aligned}$$

$$\langle \text{role} \rangle ::= [\text{'singular'}] \text{'role'} \langle \text{id} \rangle \text{'{' } \langle \text{roledef} \rangle \text{'}' [\text{' ;' }]$$

$$\begin{aligned} \langle \text{roledef} \rangle &::= \langle \text{event} \rangle [\langle \text{roledef} \rangle] \\ &| \langle \text{declaration} \rangle [\langle \text{roledef} \rangle] \end{aligned}$$

5.8.4 Events

$$\begin{aligned} \langle \text{event} \rangle &::= \text{'recv'} \langle \text{label} \rangle \text{'('} \langle \text{from} \rangle \text{' , ' } \langle \text{to} \rangle \text{' , ' } \langle \text{termlist} \rangle \text{')' ' ;' } \\ &| \text{'send'} \langle \text{label} \rangle \text{'('} \langle \text{from} \rangle \text{' , ' } \langle \text{to} \rangle \text{' , ' } \langle \text{termlist} \rangle \text{')' ' ;' } \\ &| \text{'claim'} [\langle \text{label} \rangle] \text{'('} \langle \text{from} \rangle \text{' , ' } \langle \text{claim} \rangle [\text{' , ' } \langle \text{termlist} \rangle] \text{')' ' ;' } \end{aligned}$$

$$\langle \text{label} \rangle ::= \text{'_'} \langle \text{term} \rangle$$

$$\langle \text{from} \rangle ::= \langle \text{id} \rangle$$

$$\langle \text{to} \rangle ::= \langle \text{id} \rangle$$

$$\langle \text{claim} \rangle ::= \langle \text{id} \rangle$$

5.8.5 Declarations

$\langle globaldeclaration \rangle ::= \langle declaration \rangle$
 $\quad | \text{ 'untrusted' } \langle termlist \rangle \text{ ';' }$
 $\quad | \text{ 'usertype' } \langle termlist \rangle \text{ ';' }$

 $\langle declaration \rangle ::= [\text{ 'secret' }] \text{ 'const' } \langle termlist \rangle [\text{ ':' } \langle type \rangle] \text{ ';' }$
 $\quad | [\text{ 'secret' }] \text{ 'fresh' } \langle termlist \rangle [\text{ ':' } \langle typelist \rangle] \text{ ';' }$
 $\quad | [\text{ 'secret' }] \text{ 'var' } \langle termlist \rangle [\text{ ':' } \langle typelist \rangle] \text{ ';' }$
 $\quad | \text{ 'secret' } \langle termlist \rangle [\langle type \rangle] \text{ ';' }$
 $\quad | \text{ 'inversekeys' '(' } \langle term \rangle \text{ ',' } \langle term \rangle \text{ ')' ';' }$
 $\quad | \text{ 'compromised' } \langle termlist \rangle \text{ ';' }$

 $\langle type \rangle ::= \langle id \rangle$

 $\langle typelist \rangle ::= \langle type \rangle \{ \text{ ',' } \langle type \rangle \}$

5.8.6 Terms

$\langle term \rangle ::= \langle id \rangle$
 $\quad | \text{ '{' } \langle termlist \rangle \text{ '}' } \langle key \rangle$
 $\quad | \text{ '(' } \langle termlist \rangle \text{ ')' }$
 $\quad | \langle id \rangle \text{ '(' } \langle termlist \rangle \text{ ')' }$

 $\langle key \rangle ::= \langle term \rangle$

 $\langle termlist \rangle ::= \langle term \rangle \{ \text{ ',' } \langle term \rangle \}$

Chapter 6

Modeling security protocols

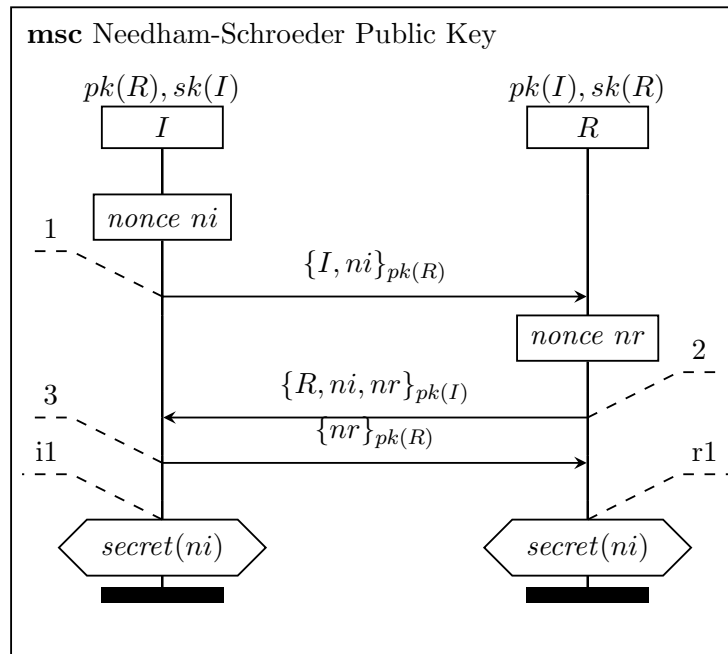


Figure 6.1: A message sequence chart description

6.1 Introduction

The correct modeling of a security protocol for analysis in the Scyther tool requires a basic understanding of the underlying symbolic model. This model is explained in detail in [2].

Roughly speaking, the symbolic analysis focuses on the following aspects:

- Logical message components and their intended function within the protocol (public versus secret, freshly generated in each run or constant)
- Message structure (pairing, encryption, signing, hashing)
- Message flow (order, involved agents)

Many other elements are abstracted away. For example, bit strings are abstracted into terms, bit strings that occur with negligible probability are abstracted away, and more complex control flow constructs such as loops are often unfolded for a (low) finite number of times.

6.2 Example: Needham-Schroeder Public Key

As an example, we show how to model a simple protocol.

Figure 6.1 depicts the Needham-Schroeder Public Key protocol. For simplicity, we have only displayed the claim by each role that the initiator nonce *ni* is secret.

We start off the protocol description by adding a multi-line comment that describes the protocol and other interesting details. Multi-line comments start with `/*` and end with `*/`.

```
/*
 * Needham-Schroeder protocol
 */
```

The protocol uses the default public/private key infrastructure: an agent *A* has a key pair `(pk(A), sk(A))`.

The protocol has two roles: the initiator role *I* and the responder role *R*. We also add a single line comment, starting with `//`.

```
// The protocol description

protocol ns3(I,R)
{
```

Scyther works with a role-based description of the protocols. We first model the initiator role. This role has two values that are local to the role: the nonce that is created by *I* and the nonce that is received. We have to declare them both.

```
role I
{
  fresh ni: Nonce;
  var nr: Nonce;
```

We now model the communication behaviour of the protocol. Needham-Schroeder has three messages, and the initiator role sends the first and last of these. Note the labels (e.g., `_1`) at the end of the `send` and `recv` keywords: these serve merely to retain the information of the connected arrows in the message sequence chart.

```
send_1(I,R, {I,ni}pk(R) );
recv_2(R,I, {ni,nr}pk(I) );
send_3(I,R, {nr}pk(R) );
```

Finally, we add the security requirements of the protocol. Without such claims, Scyther does not know¹ what needs to be checked.

Here we have chosen to check for secrecy of the generated and received nonce, and will check for non-injective agreement and synchronisation.

```
claim_i1(I,Secret,ni);
claim_i2(I,Secret,nr);
claim_i3(I,Niagree);
claim_i4(I,Nisynch);
}
```

This completes the specification of the initiator role.

For this simple protocol, the responder role is very similar to the initiator role². In fact, there are only a few differences:

1. The keywords `var` and `fresh` have swapped places: `ni` was created by *I* and a freshly generated value there, but for the role *R* it is the received value and thus a variable.

¹If you are unsure about the claims, you can also use the `--auto-claims` switch to automatically generate these at run-time.

²In general, the transformation is not that simple, but for many protocols this will suffice.

2. The keywords **send** and **recv** have swapped places.
3. The claims should have unique labels, so they have changed, and the role executing the claim is now **R** instead of **I**.

The complete role description for the responder looks like this:

```
role R
{
  var ni: Nonce;
  fresh nr: Nonce;

  recv_1(I,R, {I,ni}pk(R) );
  send_2(R,I, {ni,nr}pk(I) );
  recv_3(I,R, {nr}pk(R) );

  claim_r1(R,Secret,ni);
  claim_r2(R,Secret,nr);
  claim_r3(R,Niagree);
  claim_r4(R,Nisynch);
}
```

The full protocol description file for the *Needham-Schroeder* protocol is given in Appendix A.

Chapter 7

Specifying security properties

The Scyther framework supports a fixed set of security properties, which are specified by inserting claim events in the role specifications. The claim events are used to model the intended security properties of the protocol.

Recall that the default Scyther threat model is a network attacker that completely controls the network, and can compromise agents in various ways. The security properties that are supported by the Scyther framework are defined with respect to this threat model. For example, a secrecy claim is true if the term is not known to the adversary in any trace of the protocol execution, and false otherwise.

7.1 Specifying secrecy

The simplest security property is secrecy. To specify that a term, e.g., a nonce n , is secret at some point, insert the following claim in the role specification:

```
claim(Role, Secret, n);
```

This claim is true if the term n is not known to the adversary in any trace of the protocol execution when this claim is reached, and false otherwise. It is important to note that the role and position of the claim matter: if the claim is reached in a trace, then the term must be secret at that point. If the claim is not reached in a trace, then the claim is vacuously true. Similarly, a nonce may be secret from the perspective of one role, but not from the perspective of another role.

For example, in the Needham-Schroeder protocol, the initiator nonce ni is secret from the perspective of the initiator role I , but not from the perspective of the responder role R , because R receives ni in the second message.

7.2 Specifying authentication properties

Authentication properties are more complex than secrecy properties, and there are several different types of authentication properties that can be specified in the Scyther framework. The most basic type of authentication property is aliveness, which states that if a claim is reached in a trace, then the corresponding role must have been executed in that trace. More complex authentication properties include non-injective synchronisation, non-injective agreement, and agreement over data. To understand the subtle differences between these, we refer the reader to [2, 3, 8].

7.2.1 Aliveness

```
claim(Role, Alive);
```

7.2.2 Non-injective synchronisation

```
claim(Role, Nisynch );
```

7.2.3 Non-injective agreement

```
claim(Role, Niagree);
```

7.2.4 Agreement over data

In order to specify data agreement, e. g., that the role I agrees with the role R on a set of terms, e. g., the nonces ni and nr , one inserts two claims:

1. At the end of the I role, insert `claim(I,Commit,R,ni,nr);`
2. In the R , just before the last send (in case of a protocol with multiple roles: the last send that causally precedes the claim in the I role), insert `claim(R,Running,I,ni,nr);`

For an example of the use of these claims, see the “ns3.spdl” input file in the Scyther distribution. For a formal definition of the signals, see [8].

Chapter 8

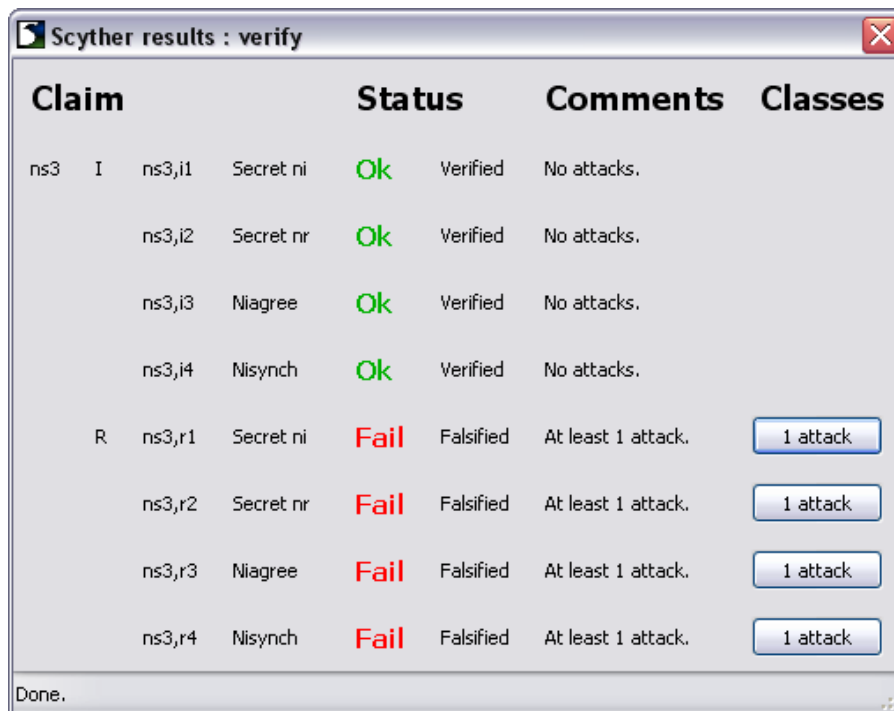
Using the Scyther tool GUI

The Scyther tool can be used in two main ways. First, through the graphical user interface (GUI) and second, through the command-line interface. For most users the first option is preferred.

In this section we detail the Scyther output when used through the GUI.

8.1 Results

As shown before, verifying the Needham-Schroeder public key protocol yields the following results as in Figure 8.1.



The image shows a window titled "Scyther results : verify" with a close button in the top right corner. The window contains a table with four columns: Claim, Status, Comments, and Classes. The table lists eight claims, grouped by role (ns3, I and R). The first four claims (ns3,i1 to ns3,i4) are for the initiator role and all have a status of "Ok". The next four claims (ns3,r1 to ns3,r4) are for the responder role and all have a status of "Fail". Each "Fail" entry has a button labeled "1 attack" next to it. The bottom of the window shows the text "Done.".

Claim	Status	Comments	Classes
ns3 I ns3,i1 Secret ni	Ok	Verified	No attacks.
ns3,i2 Secret nr	Ok	Verified	No attacks.
ns3,i3 Niagree	Ok	Verified	No attacks.
ns3,i4 Nisynch	Ok	Verified	No attacks.
R ns3,r1 Secret ni	Fail	Falsified	At least 1 attack. 1 attack
ns3,r2 Secret nr	Fail	Falsified	At least 1 attack. 1 attack
ns3,r3 Niagree	Fail	Falsified	At least 1 attack. 1 attack
ns3,r4 Nisynch	Fail	Falsified	At least 1 attack. 1 attack

Done.

Figure 8.1: Example Scyther results for a version of the Needham-Schroeder protocol. Some models of the same protocol may include different claim events, leading to a slightly different output.

The interpretation is as follows: all the claims of the initiator role `ns3,I` are correct for an unbounded number of runs.

Unfortunately, all the claims of the responder role are false. Scyther reports that it found at least one attack for each of those four claims. We could choose to view these attacks: this will be shown in Section 8.3.

In the result window, Scyther will output a single line for each claim. The line is divided into several columns. The first column shows the protocol in which the claim occurs, and the second shows

the role. In the third column a unique claim identifier is shown, of the form p, l , where p is the protocol and l is the claim label.¹ The fourth column displays the claim type and the claim parameter.

Under the header **Status** we find two columns. The fifth column gives the actual result of the verification process: it will yield **Fail** when the claim is false, and **Ok** when the claim is correct. The sixth column refines the previous statement: in some cases, the Scyther verification process is not complete (which will be explored in more detail in the next section). If this column states **Verified**, then the claim is provably true. If the column states **Falsified**, then the claim is provably false. If the column is empty, then the statement of fail/ok depends on the specific bounds setting.

The seventh column, **Comments**, serves to explain the status of the results further. In particular, the column contains a single sentences. We describe the possible results below.

- **At least X attack(s)**

Some attacks were found in the state space: however, due to the undecidability of the problem, or because of the branch and bound structure of the search, we cannot be sure that there are no other attack states.

In the default setup, Scyther will stop the verification process after an attack is found.

- **Exactly X attack(s)**

Within the statespace, there are exactly this many attacks, and no others.

- **At least X pattern(s)**

- **Exactly X pattern(s)**

These correspond exactly to the previous two, but occur in case of a ‘**Reachable**’ claim. Thus, the states that are found are not really attacks but classes of reachable states.

- **No attacks within bounds**

No attack was found within the bounded statespace, but there can possibly be an attack outside the bounded statespace.

- **No attacks**

No attack was found within the (bounded or unbounded) statespace, and a proof can be constructed that there is no attack even when the statespace is unbounded. Thus, the security property has been successfully verified.

Note that because of the nature of the algorithm, this result can even be obtained when the statespace is bounded.

8.2 Bounding the statespace

During the verification process, the Scyther tool explores a proof tree that covers all possible protocol behaviours. The default setting is to *bound* the size of this tree in some way, ensuring that the verification procedure terminates. However, importantly, even if the size of this proof tree is bounded, unbounded verification may still be achieved.

In most cases, the verification procedure will terminate and return results before ever reaching the bound. However, if the verification procedure reaches the bound, this is reported in the result window, e.g.:

No attack within bounds

¹This includes the protocol name, which is important when doing multi-protocol analysis.

This should be interpreted as: Scyther did not find any attacks, but because it reached the bound, it did not explore the full tree, and it is possible that there are still attacks on the protocol.

The default way of bounding the *maximum number of runs*, or protocol instances. This can be changed in the **Settings** tab of the main window. If the maximum number of runs is, e.g., 5, and Scyther reports **No attack within bounds**, this means that there exist no attacks that involve 5 runs or less. However, there might exist attacks that involve 6 runs or more.

For some protocols, increasing the maximum number of runs can lead to complete results (i.e. finding an attack or being sure that there is no attack), but for other protocols the result will always be **No attack within bounds**.

Note that the verification time usually grows exponentially with respect to the maximum number of runs.

8.3 Attack graphs

In Figure 8.3 we show an attack window in more detail.

The basic elements are arrows and several kinds of boxes. The arrows in the graph represent ordering constraints (caused by the prefix-closedness of events in the protocol roles, or by dependencies in the intruder knowledge). The boxes represent creation of a run, communication events of a run, and claim events.

8.3.1 Visual Notation Reference

This section provides a complete reference for understanding the visual elements in attack graphs.

Arrows and Message Flow

Arrows in attack graphs represent causality and message dependencies:

Bold green arrows Indicate *exact communication* where a message is sent and received exactly as specified in the protocol, without intruder interference or modification. These are drawn in bold with color `forestgreen`.

Black arrows All other arrows (default style). This includes:

- Messages intercepted by the intruder
- Messages from the intruder to agents
- Messages between intruder operations
- Dependencies between events

Thick vertical downward arrows typically indicate the flow of time and the sequence of events within a protocol run. Horizontal and diagonal arrows represent message transmission or dependencies. When a message path goes through an orange node (see below), the message has been modified or redirected by the intruder.

Event Boxes

Different types of events are represented by different box styles:

Rectangular box (run header) Marks the *creation of a protocol run* (role instance). Contains:

- Run number (e.g., `Run #0`, `Run #1`)
- Agent executing the role (e.g., `Agent2 in role I`)
- Agent's beliefs about communication partners (e.g., `I → Agent2`, `R → Eve`)
- Fresh values generated by this run (e.g., `ni#0`, `nr#1`)

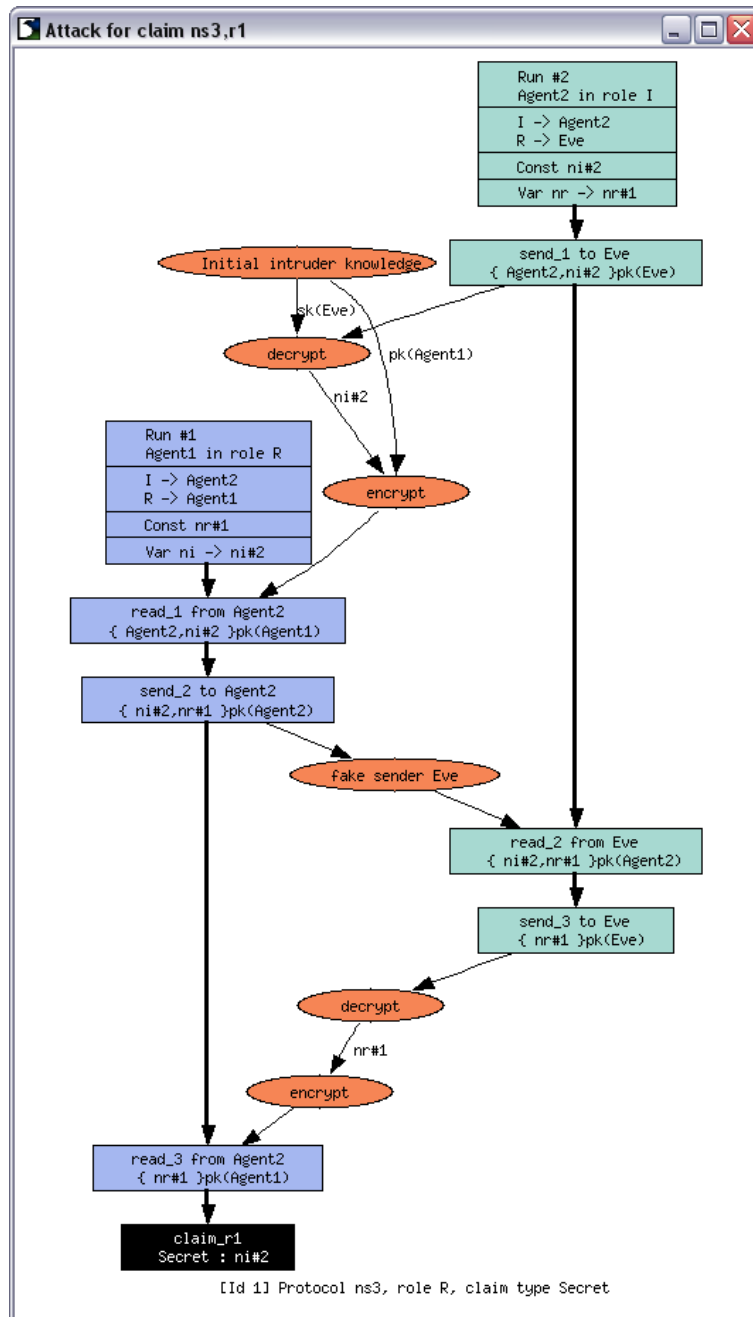


Figure 8.2: Scyther attack window

- Variable instantiations

Rectangular box (send event) Represents a *send event*. Shows:

- Event label (e.g., `send_1`)
- Sender and intended receiver
- Message content

Reading tip: The intended receiver in a send event may differ from where the message actually goes. The intruder always intercepts all messages.

Rectangular box (receive event) Represents a *receive event*. Shows:

- Event label (e.g., `recv_2`)

- Believed sender and actual receiver
- Message content as understood by the receiver

Reading tip: Compare the received message with the sent message at the arrow's source to understand intruder manipulation.

Orange node (intruder operation or redirect) Represents an *intruder action*. These nodes are filled with an orange color. They include:

- **Initial knowledge:** Shown as `M` or similar, representing terms the intruder knows from the start (agent names, constants, compromised keys)
- **Construction operations:** Labeled `encrypt`, `decrypt`, or `apply`, showing cryptographic operations the intruder performs
- **Redirect nodes:** When a message is modified between send and receive, an intermediate orange node appears with labels like:
 - `select` – extracting a subterm from a message
 - `redirect to` – changing the intended recipient
 - `fake sender` – changing the apparent sender
 - `combine` – other modifications

Colored box (claim event) Represents a *security claim*. The claim box shows:

- Claim type (`Secret`, `Nisynch`, etc.)
- Claimed property and parameters
- Whether the claim succeeded or failed

A failed claim in an attack graph indicates *this is the security property that was violated*.

Agent and Term Notation

Agent#N Represents an agent in the protocol. The number after `#` is just an identifier to distinguish different agents. Example: `Agent#0`, `Agent#1`.

ni#N, nr#N, nk#N Nonces (random values) with their generating run number. `ni#2` means “nonce `ni` generated by run `#2`”. This notation helps track which run created which values.

{term}key Encryption of `term` with `key`. Examples:

- `{Na}pk(Bob)` - `Na` encrypted with Bob's public key
- `{M}k(A,B)` - `M` encrypted with shared key between `A` and `B`

(term1, term2) Tuple or pairing of multiple terms. Represents concatenation or bundling of message components.

Reading Attack Graphs: A Strategy

To effectively understand an attack graph:

1. **Identify the failed claim:** Look for the highlighted claim event, which is often at the bottom. This is what the attack violates.
2. **Count the runs:** See how many protocol instances are involved. Most attacks use 2-3 runs.
3. **Trace backwards from the failed claim:** Follow arrows backwards to see what events led to the security violation.
4. **Examine arrows and nodes:**

- Many green arrows? The attack may exploit protocol logic rather than message manipulation.
 - Black arrows through orange nodes? The intruder is modifying or redirecting messages.
 - Orange **encrypt/decrypt** nodes? The intruder is actively constructing new messages.
5. **Check agent beliefs:** In run headers, compare who the agent thinks they're talking to versus who is actually in other runs.
 6. **Track nonce origins:** Follow nonce identifiers (e.g., **ni#2**) to see which run created them and where they're being used.
 7. **Identify the attack step:** Look for the critical point where:
 - A message is redirected or modified (orange redirect node in the arrow path)
 - The intruder constructs a new message (orange **encrypt/decrypt** node)
 - An agent accepts a message with wrong beliefs

Common Attack Patterns

Recognizing these patterns helps quickly understand attack graphs:

Man-in-the-middle Two runs where each agent thinks they're talking to the other, but messages pass through the intruder. Look for: two runs with swapped beliefs, mismatches in the data they should be agreeing on, or multiple orange redirect nodes.

Reflection attack An agent receives back their own message. Look for: a message sent by run N and received by the same run N, often with redirection.

Type confusion A value meant for one purpose is used for another. Look for: the same nonce or value appearing in different message contexts.

Parallel session attack Multiple concurrent runs where the intruder mixes messages between sessions. Look for: 3+ runs, messages from different runs being combined.

8.3.2 Runs

Each vertical axis represents a run (an instance of a protocol role). Thus, in this attack we see that there are two runs involved. Each run starts with a diamond shaped box. This represents the creation of a run, and is used to give information about the run.

For the run on the left-hand side in the attack we have this information:

```
Run #1
Agent2 in role I
I -> Agent2
R -> Agent1
```

Each run is assigned a run identifier (here 1), which is an arbitrary number that enables us to uniquely identify each run. This run executes the R role of the protocol. It is being executed by an agent called **Agent1**, who thinks he is talking to **Agent2**. Note that although run 2 is being executed by **Agent2**, this agent does not believe he is talking to **Agent1**.

```
Run #2
Agent2 in role I
I -> Agent2
R -> Eve
```

In the run on the right, we see This run represents an instance of the role I. From the second line we can see which agent is executing the run, and who he thinks he is talking to. In this example, the run is executed by an agent called **Agent2**, who thinks the responder role is being executed by the untrusted agent **Eve**.²

Additionally, the run headers contain information on the freshly generated values (e.g., run 1 generates **nr#1**) and information on the instantiation of the local variables (e.g., run 1 instantiates its variable **ni** with the nonce **ni#2** or run 2).

8.3.3 Communication events

Send events denote the sending of a message. The first send occurs in this attack is the first send event of run 2.

```
send_1(Eve, { Agent#0, ni#2 }pk(Eve) )
```

Every time a message is sent, it is effectively given to the intruder. In this case, because the intruder knows the secret key **sk(Eve)** of the agent **Eve**, he can decrypt the message and learns the value of the nonce **ni#2**.

Receive events correspond to the successful reception of a message. The first receive event that can occur in this attack is the first receive event of run 0.

```
recv_1(Agent#0, { Agent#0, ni#2 }pk(Agent#1) )
```

This tells us that the agent executing this run, **Agent#1**, reads a message that is apparently coming from **Agent#1**. The message that is received is **{ Agent#0, ni#2 }pk(Agent#1)** : the name of the agent he thinks he is communicating with and the nonce **ni#2**, encrypted with his public key.

The incoming arrow does not indicate a direct sending of the message. Rather, it denotes an ordering constraint: this message can only be received *after* something else has happened. In this case, we see that the message can only be received after run 2 sends his initial message. The reason for this is the nonce **ni#2**: the intruder cannot predict this nonce, and thus has to wait until run 2 has generated it.

In the graph the connecting arrow is red and has a label “construct” with it: this is caused by the fact that the message sent does not correspond to the message that is received. We know the intruder can only construct the message to be received after the sent message, and thus it must be the case that he uses information from the sent message to construct the message that is received. Other possibilities include a green and a yellow arrow. A yellow arrow indicates that a message was sent, and received in exactly the same form: however, the agents disagree about who was sending a message to whom. It is therefore labeled with “redirect” because the intruder must have redirected the message. A green arrow (not in the picture) indicating that a message is received exactly the same as it was sent, representing a normal message communication between two agents.

Note that a **recv** event without an incoming arrow denotes that a term is received that can be generated from the initial knowledge of the intruder. There is no such event in the example, but this can occur often. For example, if a role reads a plain message containing only an agent name, the intruder can generate the term from his initial knowledge.

8.3.4 Claims

8.4 GUI Settings and Configuration

The Scyther GUI provides extensive configuration options through the **Settings** dialog, accessible from the main window. These settings impact the effectiveness and reliability of protocol verification.

²Because this agent is talking to the untrusted agent, of course all information is leaked, and no guarantees can be given.

8.4.1 Verification Parameters

The following parameters control how Scyther explores the protocol state space:

Maximum number of runs Controls how many protocol sessions (runs) Scyther will consider. Default is typically 4-5 runs.

If a claim is analyzed and the result is “verified” or “falsified”, increasing the maximum number of runs will not change the result, as the claim is provably true, or there exists a counterexample. However, if the result for a claim is “no attacks within bounds”, then increasing the maximum number of runs may yield different results:

- May find attacks that require more protocol instances
- Significantly increases verification time (typically exponentially)

Recommended values: 3-4 for quick checks, 5-7 for thorough analysis. However, until “verified” or “falsified” is reached, results are not conclusive.

Maximum search depth Limits the length of attack traces (number of events). Default is around 20-30. Increase if you suspect attacks with many steps. Very large values can cause memory issues.

Pruning Pruning controls the branch-and-bound search strategy that aims to show the user the “simplest” attack, instead of the first attack that it finds. Note that this applies per-claim. Possible values are:

- 0 = find all attacks
- 1 = find first attack
- 2 = find best attack (continues search after finding an attack to see if it can find a simpler one)

Default value is 2.

Maximum number of attacks How many attacks to find before stopping. Default is 1 (stop after first attack). Set to higher values or 0 (unlimited) to find all attacks, useful for understanding attack patterns.

8.4.2 Verification Modes

Different verification modes are available through the settings:

Standard verification Verifies all claims in the protocol file.

Automatic claims Generates and verifies standard security claims automatically for all roles, even if no explicit claims are specified. Note that while this is a quick way to get an overview of potential weaknesses, it may not capture all relevant security properties for a given protocol.

Characterization Generates the complete state-space characterization, showing all possible protocol behaviors within bounds. Users can then manually expect these to investigate if they match expectations.

8.4.3 Performance Tuning

For slow verifications:

- Reduce maximum runs (try 3-4 instead of 5+)
- Set maximum attacks to 1 (stop after finding first attack)
- Reduce maximum search depth if appropriate

For thorough analysis:

- Increase maximum runs gradually (5, 6, 7...)
- Set maximum attacks to 0 (find all)
- Monitor memory usage for large protocols

8.5 GUI Menu Reference

This section provides a complete reference for all menu options and GUI features.

8.5.1 File Menu

New Create a new empty protocol file.

Open Open an existing `.spd1` protocol file. Keyboard shortcut: `Ctrl+O` (`Cmd+O` on macOS).

Save Save the current file. Keyboard shortcut: `Ctrl+S` (`Cmd+S` on macOS).

Save As Save the current file with a new name.

Exit Close the application.

8.5.2 Verify Menu

Verify claims Verifies all explicit claim statements in the protocol. This is the most common verification mode. The tool will analyze each claim and report whether it is correct or if attacks exist.

Automatic claims Automatically generates and verifies standard security properties for all roles in the protocol, even if no explicit claims are specified. Useful for quick security assessment of a protocol.

Characterize Generates a complete state-space characterization showing all possible protocol executions within the specified bounds. This provides a comprehensive view of protocol behavior but can be computationally expensive.

8.5.3 Working with Multiple Protocols

To analyze multiple protocols simultaneously:

1. Create a new `.spd1` file
2. Use `#include` directives or paste multiple protocol definitions into the same file
3. Run verification on the combined file

This allows detection of multi-protocol attacks where an attacker exploits interactions between different protocols running in the same environment.

8.5.4 Exporting Results

Attack graphs can be saved for inclusion in papers or presentations:

- Use the system's screenshot functionality to capture attack windows
- For programmatic access, use the `--dot-output` command-line option (see Section 9)
- The generated `.dot` files can be converted to PDF, PNG, or SVG using Graphviz

8.5.5 Tips for Effective Use

- Use command-line options like the `--no-splash/-N` and `--verify` flags for repeated testing to skip the splash screen, and directly provide input files for verification
- Save custom settings by modifying the preferences file
- Use automatic claims first for quick assessment, then add specific claims

Chapter 9

Using the Scyther command-line tools

All of the features offered by the Scyther GUI are also available through command-line tools. Additionally, the command-line tools offer some features that currently cannot be accessed through the GUI.

Depending on your platform, the Scyther directory contains one of the following executables:

- **Scyther/scyther-linux**
- **Scyther/scyther-w32**
- **Scyther/scyther-mac** (macOS, both Intel and Apple Silicon)

In the following, we assume that the linux version is used. If you have a different version, please replace **scyther-linux** in the below by the executable for your platform.

To get a list of (some) of the command-line options, run the executable with the **--help** switch, e.g.:

```
scyther-linux --help
```

To analyze the Needham-Schroeder protocol and generate a .dot file (the input language for the Graphviz tool) for the attacks, use

```
scyther-linux --dot-output --output=ns3-attacks.dot ns3.spdl
```

The resulting output file can then be rendered by graphviz, e.g.:

```
dot -Tpdf -O ns3-attacks.dot
```

This yields several PDF files **ns3-attacks.dot[.N].pdf** that contain the attack graphs.

To get a more complete list of command-line options, run the executable with the **--expert --help** switch, e.g.:

```
scyther-linux --expert --help
```

9.1 Python wrapper scripts

The Scyther distribution includes several Python wrapper scripts that provide alternative interfaces to the backend tool. These scripts are located in the main Scyther directory.

9.1.1 `scyther.py`: Simple command-line wrapper

The `scyther.py` script provides a simple Python-based command-line interface to the Scyther backend. It accepts the same arguments as the native backend executables.

To use `scyther.py`, run:

```
python3 scyther.py [options] [protocol.spdl]
```

For example, to analyze a protocol with specific bounds:

```
python3 scyther.py --max-runs=3 --max-length=10 ns3.spdl
```

Running `scyther.py` without arguments displays the help information:

```
python3 scyther.py
```

The script automatically detects the appropriate backend binary for your platform (Linux, Windows, or macOS) and forwards all arguments to it. This wrapper is useful for scripting and automation tasks where a consistent interface across platforms is desired.

9.1.2 `scyther-gui.py`: GUI with command-line options

While `scyther-gui.py` is primarily used to launch the graphical interface, it also supports several command-line options for automation and batch processing:

-V, --verify Immediately verify all claims in the protocol without showing the GUI. Requires an input file.

```
python3 scyther-gui.py --verify protocol.spdl
```

-s, --state-space Immediately generate the complete state-space characterization of the protocol without showing the GUI. Requires an input file.

```
python3 scyther-gui.py --state-space protocol.spdl
```

-a, --auto-claims Immediately verify the protocol using automatically generated default claims without showing the GUI. Requires an input file.

```
python3 scyther-gui.py --auto-claims protocol.spdl
```

-N, --no-splash Suppress the splash screen when launching the GUI. Useful for repeated testing or when the GUI is launched programmatically.

```
python3 scyther-gui.py --no-splash
```

-l, --license Display the license information and exit.

```
python3 scyther-gui.py --license
```

These options allow `scyther-gui.py` to be used in automated testing scripts while still benefiting from the GUI's result formatting and display capabilities. When used with verification options (`--verify`, `--state-space`, or `--auto-claims`), the results window will appear automatically with the analysis results.

9.1.3 json-scyther.py: Batch processing interface

The `json-scyther.py` script provides a JSON-based batch processing interface to Scyther. This is particularly useful for running multiple protocol analyses with different configurations or for integrating Scyther into larger automated verification pipelines.

The script reads JSON-formatted input from a file, where each line contains a tuple of:

1. A list of protocol file paths
2. Scyther options as a string
3. A filter string (typically a claim identifier)

Input format example:

```
[["protocol1.spdl", "protocol2.spdl"], "--max-runs=4", "claim_id"]
```

To use `json-scyther.py`:

```
python3 json-scyther.py input.json [line_numbers...]
```

If line numbers are specified, only those lines from the input file will be processed. Otherwise, all lines are processed sequentially.

Example usage:

```
python3 json-scyther.py batch-tests.json 1 5 10
```

This processes only lines 1, 5, and 10 from `batch-tests.json`.

The JSON interface can be used for:

- Running systematic parameter sweeps over multiple protocols
- Integrating Scyther into continuous integration pipelines
- Automating multi-protocol attack analysis
- Reproducing research experiments with documented configurations

Chapter 10

Advanced topics

10.1 Modeling more than one asymmetric key pair

Asymmetric keys are typically modeled as two functions: one function that maps the agents to their public keys, and another function that maps agents to their secret keys.

By default, each agent x has a public/private key pair $(\mathbf{pk}(x), \mathbf{sk}(x))$.

To model other asymmetric keys, we first define the two functions, which are for example named $\mathbf{pk2}$ for the public key function, and $\mathbf{sk2}$ for the secret key function.

```
const pk2: Function;  
secret sk2: Function;
```

We also declare that these functions represent asymmetric key pairs:

```
inversekeys (pk2,sk2);
```

If defined in this way, a term encrypted with $\mathbf{pk2}(x)$ can only be decrypted with $\mathbf{sk2}(x)$ and vice versa.

10.2 Approximating equational theories

The operational semantics underlying Scyther currently only consider syntactic equality: two (ground) terms are equal if and only if they are syntactically equivalent. However, there are several common cryptographic constructions that are more naturally modeled by using certain equalities. For example:

1. $g^{ab} \pmod{N}$ and $g^{ba} \pmod{N}$, to model Diffie-Hellman exponentiation.
2. $k(A, B)$ and $k(B, A)$, to model bidirectional long-term keys.

Although Scyther does not provide direct support for such equational theories, there exists a straightforward underapproximation.

The core idea is that instead of modeling the term equality, we provide the adversary with the ability to learn all terms in an equivalence class if he learns one of its elements. For example, for the equivalence class $\{k(A, B), k(B, A)\}$ we can provide the adversary with the ability to learn $k(B, A)$ from $k(A, B)$, and vice versa. We can model this by introducing an appropriate helper protocol (denoted by the prefix '@'):

```
protocol @keysymmNaive(X) {  
  role X {  
    var Y: Agent;  
  
    recv_!1(X,X, k(X,Y) );  
    send_!2(X,X, k(Y,X) );  
  }  
}
```

Because the role can be instantiated for any agents X and Y , this covers all possible combinations of agents.

The above naive approximation can be significantly improved. One obvious and practically relevant omission is that the adversary usually learns encrypted messages, but not the key. In such cases, we still would like to model that $\{ m \}_{k(A,B)} = \{ m \}_{k(B,A)}$. Thus we adapt our helper protocol:

```
protocol @keysymmInefficient(X,Y) {
  role X {
    var Y: Agent;

    recv_!1(X,X, k(X,Y) );
    send_!2(X,X, k(Y,X) );
  }
  role Y {
    var X: Agent;
    var m: Ticket;

    recv_!1(Y,Y, { m }k(X,Y) );
    send_!2(Y,Y, { m }k(Y,X) );
  }
}
```

If the protocol contains further terms in which the symmetric keys appear in other positions, such as in nested encryptions or hashes, we would add further roles.

The above approximation is often inefficient in practice. We can improve performance by making the helper protocol rules more tight, i. e., by exploiting more type information about the protocol. For example, if the protocol transmits two types of encrypted messages:

1. $\{ I, nI, nR \}_{k(I,R)}$, and
2. $\{ nI \}_{k(I,R)}$,

then we would modify the helper protocol in the following way:

```
protocol @keysymm(X,Y,Z) {
  role X {
    var Y: Agent;

    recv_!1(X,X, k(X,Y) );
    send_!2(X,X, k(Y,X) );
  }
  role Y {
    var X,Z: Agent;
    var n1,n2: Nonce;

    recv_!1(Y,Y, { Z,n1,n2 }k(X,Y) );
    send_!2(Y,Y, { Z,n1,n2 }k(Y,X) );
  }
  role Z {
    var X,Y: Agent;
    var n1: Nonce;

    recv_!1(Z,Z, { n1 }k(X,Y) );
    send_!2(Z,Z, { n1 }k(Y,X) );
  }
}
```

In general, one would manually inspect the protocol and extract all positions in which a term from an equivalence class occurs as a subterm. For each of these positions, we model an appropriate role in the helper protocols.

This is also used to model, for example, Diffie-Hellman exponentiation. For exponentiation we introduce an abstract function symbol, e.g., `exp`, and a public constant `g`. We then introduce a helper protocol with roles to model that $\text{exp}(\text{exp}(g, X), Y) = \text{exp}(\text{exp}(g, Y), X)$.

In practice, this type of underapproximation has proven to be extremely effective, to the point that all known attacks on real-world protocols that can be modeled using the “real” equational theory, are found by Scyther when using the underapproximation.

One caveat is that while this approximation works well for secrecy and data-agreement, it can cause message-based agreement properties (such as synchronisation) to fail, because their message equality checks are syntactical. These checks are not affected by the introduction of helper protocols.

10.3 Modeling time-stamps and global counters

Scyther’s underlying protocol model currently does not provide support for variables that are shared among the runs of an agent. Effectively, each run starts with a “clean slate”, independent of any runs that have been executed previously. In other words, globally update state can not be modeled directly.

In the following sections we provide some modeling approaches for common problems.

10.3.1 Modeling global counters

Globally incremented counters can be modeled using freshly generated values. This ensures that each run uses a different value. The model is coarse in the sense that the recipient of such a counter cannot check that it is the successor of the previous value of the counter.

10.3.2 Modeling time-stamps using nonces

There are at least two ways to model time-stamps.

The first model is more appropriate for protocols where the probability that a given time-stamp value is accepted by two runs is very low. This occurs when time-stamps have great precision or when two runs occur only sequentially, possibly with some delay time in between. In this case, one can model time-stamps as freshly generated values, e.g., nonces. To cater for the fact that the adversary typically knows the time (and thus can also predict time-stamps), we prepend a send event to the role that provides the adversary with the value of the time-stamp that will be used. For example, we would prepend the send with label `!T1` for time-stamp `T1` as in the following example:

```
usertype Timestamp;

protocol MyProtocol(Server,Client) {
  role Server{
    fresh T1: Timestamp;

    /* Time-stamps are unique per run */
    send_!T1(Server, Server, T1);

    ...
    /* Server uses time-stamp value */
    send_2(Server,Client, { Server, T1 }pk(Client) );
    ...
  }
}
```

10.3.3 Modeling time-stamps using variables

The second model is more appropriate when it is reasonable that two runs may accept the same time-stamp value. This is common for coarse time-stamps, or for roles that are typically executed with high parallelism, such as server roles. In such cases, one can instead model timestamps as values that

are determined by the adversary. In contrast to the previous solution, this is done by prepending a receive event. For example:

```
usertype Timestamp;

protocol MyProtocol(Server,Client) {
  role Server{
    var T1: Timestamp;

    /* Adversary chooses time-stamp value */
    recv_!T1(Server, Server, T1);

    ...
    /* Server uses time-stamp value */
    send_2(Server,Client, { Server, T1 }pk(Client) );
    ...
  }
}
```

10.4 Multi-protocol attacks

Scyther can be used to check for so-called *multi-protocol attacks* (closely related concepts are *cross-protocol attacks* and *chosen protocol attacks*). These attacks depend on the interactions between different (sub)protocols: sometimes the adversary can use messages or message components from one protocol to attack another. For more information on this type of attack we refer to [6, 7].

The easiest way to check for multi-protocol attacks in Scyther is to combine two protocol descriptions into a single file, i.e., create a new `.spd1` file and paste into this file two other `.spd1` files. The resulting file models an environment in which both protocols are running. Use Scyther to evaluate the claims in the combined file.

Chapter 11

Troubleshooting and FAQ

This chapter addresses common issues, questions, and best practices for using Scyther effectively.

11.1 Installation and Setup Issues

11.1.1 macOS: “Binary cannot be opened because the developer cannot be verified”

When downloading the Scyther binary on macOS, the system may block execution due to Gatekeeper security features.

Solution: Remove the quarantine attribute from the binary:

```
xattr -d com.apple.quarantine Scyther/scyther-mac
```

Alternatively, right-click (or Control-click) the binary in Finder and select “Open” the first time you run it.

11.1.2 GUI fails to start or shows blank windows

The Scyther GUI requires wxPython 4.0 or later.

Check your Python version and wxPython installation:

```
python3 --version  
python3 -c "import wx; print(wx.version())"
```

Install or update wxPython:

```
pip3 install --upgrade wxPython
```

On some systems, you may need to use a virtual environment or install additional system dependencies. See the `INSTALL.md` file in the `gui/` directory for platform-specific instructions.

11.1.3 Command not found or Python errors

Scyther requires Python 3. If you see “command not found” or syntax errors:

- Ensure Python 3 is installed: `python3 --version`
- Use `python3` instead of `python` to run scripts
- Check that all dependencies are installed: `pip3 install -r requirements.txt`

11.2 Understanding Verification Results

11.2.1 What’s the difference between “No attacks within bounds” and “No attacks”?

No attacks within bounds Scyther explored the state space up to the configured limits (e.g., maximum 5 runs, maximum depth 20) and found no attacks, but also no proof. Hence, attacks involving more runs or longer traces might still exist.

Action: Gradually increase the maximum number of runs in the settings. Try 6, 7, or 8 runs. If still no attacks are found and the result remains “within bounds,” your protocol likely meets the property for practical scenarios.

No attacks Scyther has proven that no attacks exist on the property in the model, even for unbounded numbers of runs. This is a complete verification result.

Meaning: Your protocol is secure for the specified claims under the Dolev-Yao attacker model and Scyther’s semantics.

11.2.2 Verification is taking too long

If verification doesn’t complete in reasonable time:

1. **Reduce maximum runs:** Try 3 or 4 instead of 5+. Most attacks are found with few runs.
2. **Limit attack search:** Set maximum attacks to 1 (stop after finding first attack).
3. **Check protocol complexity:** Protocols with many roles, long message sequences, or complex terms take longer to verify.
4. **Try automatic claims first:** If you have many explicit claims, try verifying with automatic claims to get quick feedback.

11.2.3 Why did Scyther find an attack I don’t think is realistic?

Scyther uses the Dolev-Yao attacker model, which makes certain assumptions:

- **Perfect cryptography:** Encryption cannot be broken without the key
- **Attacker controls the network:** The attacker can intercept, modify, and inject messages
- **Nonces are perfectly random:** Cannot be guessed by the attacker

If an attack seems unrealistic:

1. Check if it relies on assumptions not in your threat model
2. Consider if additional checks or type distinctions should be added to the protocol
3. Verify that your protocol specification matches the intended implementation
4. Remember: Even “unrealistic” attacks may reveal design weaknesses worth addressing

11.3 Protocol Modeling Issues

11.3.1 No attacks found but I expected one

Common causes:

1. **Modeling errors:** Review your protocol specification for syntax errors, incorrect term construction, or missing messages. Try automatic claims and in particular reachability claims: if a reachability claim is not satisfied, there is likely a modeling error.
2. **Insufficient runs:** The attack requires more protocol instances than your bound. Increase maximum runs to 6-8.
3. **Wrong claim type:** Make sure you're using the right claim type (e.g., `Secret` vs `Nisynch`).
4. **Over-constrained roles:** Check that role variables aren't unnecessarily constrained. The attacker should be able to execute roles with different agents.

11.3.2 Parse errors and syntax issues

Common syntax mistakes:

- Missing semicolons after statements
- Mismatched braces `{ }` or parentheses `( )`
- Undefined variables (all variables must be declared as `var` or `fresh`)
- Wrong term construction (check encryption syntax: `{term}key`)
- Case sensitivity (Scyther is case-sensitive)
- Comments not properly closed `(/* ... */)`

Tip: Look at example protocols in the `Protocols/` directory for correct syntax patterns.

11.3.3 Unexpected variable bindings

Variables in roles can be bound in ways you might not expect:

- A role can run with any agent, including the intruder
- Variables are bound when a message is received
- The same agent can play multiple roles (including communicating with themselves)

To constrain role execution, use match statements or add inequalities in global declarations.

11.4 Common Questions

11.4.1 Should I use automatic claims or write my own?

Use automatic claims when:

- Getting started with a new protocol
- Doing quick security assessment
- You want standard authentication and secrecy properties

Write explicit claims when:

- You need specific, non-standard properties
- You want fine-grained control over what's verified
- You're analyzing a variant and want to compare specific claims
- Writing formal proofs that reference specific claims

Best practice: Use automatic claims first to understand the protocol, then add explicit claims for specific properties you care about.

11.4.2 Can Scyther verify protocols with Diffie-Hellman or other cryptographic operations?

Scyther supports standard Dolev-Yao operations (encryption, hashing, pairing). For advanced cryptographic operations like Diffie-Hellman, XOR, or exclusive-or more advanced tools like the Tamarin prover [1, 9] are more appropriate.

However, you can approximate some of these operations in Scyther using helper protocols (see Section 10).

11.4.3 How do I verify a protocol with more than two roles?

Scyther handles protocols with any number of roles. Simply define all roles within the same protocol:

```
protocol ThreeParty(Alice,Bob,Server)
{
  role Alice { ... }
  role Bob { ... }
  role Server { ... }
}
```

Each role can have its own claims. Scyther will consider all possible interleavings of the roles.

11.4.4 Where can I find more example protocols?

Example protocols are in the `Protocols/` directory:

- `Protocols/` - Standard protocols (Needham-Schroeder, Kerberos, TLS, etc.)
- `gui/Protocols/` - Additional protocol examples
- The examples cover various patterns: challenge-response, key exchange, authenticated key establishment

Study these examples to learn modeling patterns and best practices.

Bibliography

- [1] David A. Basin, Cas Cremers, Jannik Dreier, and Ralf Sasse. *Modeling and Analyzing Security Protocols with Tamarin - A Comprehensive Guide*. Information Security and Cryptography. Springer, 2026. <https://tamarin-prover.com/book/index.html>.
- [2] Cas Cremers and Sjouke Mauw. *Operational Semantics and Verification of Security Protocols*. Information Security and Cryptography. Springer, 2012. <https://doi.org/10.1007/978-3-540-78636-8>.
- [3] Cas J. F. Cremers, Sjouke Mauw, and Erik P. de Vink. Injective synchronisation: An extension of the authentication hierarchy. *Theor. Comput. Sci.*, 367(1-2):139–161, 2006.
- [4] C.J.F. Cremers. Scyther source code repository. <https://github.com/cascremers/scyther/>.
- [5] C.J.F. Cremers. The Scyther tool: Automated verification of security protocols. <https://people.cispa.io/cas.cremers/scyther/>.
- [6] C.J.F. Cremers. Feasibility of Multi-Protocol Attacks. In *Proc. of The First International Conference on Availability, Reliability and Security (ARES)*, pages 287–294, Vienna, Austria, April 2006. IEEE Computer Society.
- [7] J. Kelsey, B. Schneier, and D. Wagner. Protocol interactions and the chosen protocol attack. In B. Christianson, B. Crispo, T.M.A. Lomas, and M. Roe, editors, *Fifth International Workshop on Security Protocols*, volume 1361 of *LNCS*, pages 91–104. Springer, 1997.
- [8] G. Lowe. A Hierarchy of Authentication Specifications. In *Proc. 10th IEEE Computer Security Foundations Workshop (CSFW)*, pages 31–44. IEEE, 1997.
- [9] Tamarin Prover Team. Tamarin Prover. <https://tamarin-prover.com/>.

Appendix A

Full example specification for Needham-Schroeder public key

```
/*
 * Needham-Schroeder protocol
 */

// The protocol description

protocol ns3(I,R)
{
  role I
  {
    fresh ni: Nonce;
    var nr: Nonce;

    send_1(I,R, {I,ni}pk(R) );
    recv_2(R,I, {ni,nr}pk(I) );
    claim(I,Running,R,ni,nr);
    send_3(I,R, {nr}pk(R) );

    claim_i1(I,Secret,ni);
    claim_i2(I,Secret,nr);
    claim_i3(I,Alive);
    claim_i4(I,Weakagree);
    claim_i5(I,Commit,R,ni,nr);
    claim_i6(I,Niagree);
    claim_i7(I,Nisynch);
  }

  role R
  {
    var ni: Nonce;
    fresh nr: Nonce;

    recv_1(I,R, {I,ni}pk(R) );
    claim(R,Running,I,ni,nr);
    send_2(R,I, {ni,nr}pk(I) );
    recv_3(I,R, {nr}pk(R) );

    claim_r1(R,Secret,ni);
    claim_r2(R,Secret,nr);
    claim_r3(R,Alive);
    claim_r4(R,Weakagree);
    claim_r5(R,Commit,I,ni,nr);
  }
}
```

```
    claim_r6(R,Niagree);  
    claim_r7(R,Nisynch);  
  }  
}
```

Index

@, 22
!, 20

abbreviate, 23
Agent, 19
agent notation, 37
agreement (on messages), 20
agreement on data, 20, 32
Alive, 20
arrow colors, 35
asymmetric key
 multiple pairs per agent, 47
asymmetric keys, 18
at least X attack(s), 34
at least X pattern(s), 34
atomic term, 17
attack graph, 35
 notation, 35
 reading strategy, 37
attack graphs
 export, 41
attack patterns, 38
attack window, 35
automatic claims, 41
automatic claims vs explicit claims, 53

batch processing, 45
bidirectional keys, 47
black arrows, 35
BNF, 24
bounded vs unbounded verification, 52
box types, 35

case-sensitive, 17
characterization, 41
chosen protocol attacks, 50
claim, 20
claim event, 20, 37
command-line tools, 43
comments, 17
Commit, 20, 32
common attacks, 38
communication event, 39
const, 22
constant, 17
construct, 39
counter, *see* global counter

cross-protocol attacks, 50
cryptographic operations, 54

data agreement, 20, 32
define macro, 23
Diffie-Hellman, 54
Diffie-Hellman exponentiation, 47
Dolev-Yao model, 52

Empty, 20
encryption notation, 37
equational theories, 47, 54
event
 claim, 20
 match, 21
 not match, 21
 recv, 19
 send, 19
event boxes, 35
events, 19
exactly X attack(s), 34
exactly X pattern(s), 34
example protocols, 54
exponentiation, *see* Diffie-Hellman exponentiation
exporting results, 41

failed claim, 37
false negatives, 52
FAQ, 51
File menu, 41
frequently asked questions, 51
freshly generated value, 17
Function, 19

global constant, 17
global counter, 49
global declarations, 22
green arrows, 35
GUI, 33
 menu reference, 41
 settings, 39
 startup problems, 51
 tips, 42
 using Scyther without GUI, 43

hash functions, 19
hashfunction, 19

- helper protocol, **22**, 47
- identifier, 17
- import file, 23
- include**, 23
- input file, 23
- internal computation events, 20
- intruder event, 37
- json-scyther.py, 45
- k(X,Y)**, 18
- loading files from command line, 13
- macOS
 - Gatekeeper, 51
- macro**, 23
- man-in-the-middle attack, 38
- match event, 21
- max-length, 40
- maximum attacks, 40
- maximum runs, 40
- maximum search depth, 40
- message agreement, 20
- message flow, 35
- modeling errors, 52
- multi-protocol analysis, 41
- multi-protocol attacks, 50
- multi-role protocols, 54
- multiple asymmetric key pairs per agent, 47
- multiple roles per agent, 24
- Needham-Schroeder protocol, 28
- Niagree**, 20
- Nisynch**, 20
- no attacks, 34
- no attacks within bounds, 34, 52
- non-injective agreement, 20, 32
- non-injective synchronisation, 20
- Nonce**, 19
- nonce, 17
- nonce notation, 37
- not match event, 21
- NS, *see* Needham-Schroeder protocol
- one role per agent, 24
- one-role-per-agent**, 24
- orange node, 37
- pairing, 18
- pairing notation, 37
- parallel session attack, 38
- parse errors, 53
- pattern match events, 20
- performance issues, 52
- performance tuning, 40
- pk(X)**, 18
- protocol definition, 21
- protocol library, 54
- pruning, 40
- Python
 - version requirements, 51
 - wrapper scripts, 43
- quick start tutorial, 13
- random value, 17
- Reachable**, 20
- receive box, 36
- receive event, 36
- recv**, 19
- redirect node, 37
- reflection attack, 38
- role definition, 21
- role matching, 53
- run, 38
- run header, 35
- Running**, 20, 32
- scyther-gui.py
 - command-line options, 44
- scyther.py, 44
- Secret**, 20
- security properties, 20
- security warnings, 51
- send**, 19
- send box, 36
- send event, 36
- settings dialog, 39
- sk(X)**, 18
- symmetric keys, 18
- symmetric-role protocol, **22**
- synchronisation, 20
- syntactic equality, 47
- syntax errors, 53
- term notation, 37
- three-party protocols, 54
- Ticket**, 19
- time-stamps, 49–50
- troubleshooting, 51
- tupling, 18
- type confusion attack, 38
- understanding attacks, 37
- unrealistic attacks, 52
- usertype**, 19
- var, 18
- variable, 18

- variable binding, 53
- verification, 34
 - slow, 52
- verification modes, 40
- verification parameters, 40
- Verify menu, 41
- visual notation, 35

- Weakagree**, 20
- whitespace, 17
- wxPython, 51